



คู่มือสรุปคัดกรองเนื้อหาเตรียมสอบวิชา Embedded System Design

สรุปเฉพาะประเด็นเน้นออกสอบ 100% สกัดจากตำรา AVR Microcontroller Ch 1-6 & Microprocessors

✗ ส่วนที่ตัดออก (ส่วนที่ไม่จำเป็นต้องท่องจำ / ไม่เกี่ยวกับข้อสอบ):

- ประวัติศาสตร์คอมพิวเตอร์ยุคแรก: หลอดสุญญากาศ Vacuum Tubes, ทรานซิสเตอร์ยุค 1950s และประวัติชิป Intel 4004/8080
- ฟิลิ์กส์สารกึ่งตัวนำ & ไฟฟ้าลิก: โครงสร้างทรานซิสเตอร์ MOSFET level, Gate Capacitance (Cin), Silicon Doping, อัตราการคายความร้อน (Watts/BTU)
- โปรโตคอลภายนอกที่ไม่ได้ออกสอบ: CAN Bus, USB 2.0 Packet Framing, Ethernet MAC Controller, ZigBee, I2S Audio
- รายละเอียด GUI ซอฟต์แวร์ย่อย: ขั้นตอนลงทะเบียน Atmel Studio VSIX Extension Plugin, Visual Studio Extensibility SDK

✓ ส่วนที่ต้องเน้น 100% (จุดเน้นออกสอบจริงแยกตาม Part 1 & Part 2):

PART 1: สรุปเน้นสอบตอนเช้า (ปิดตำรา - CLOSED-BOOK EXAM)

1.1 พีชคณิตบูลีน & K-Map 4 ตัวแปร (Gate Minimization)

- กฎ DeMorgan: $(A \cdot B)' = A' + B'$ และ $(A + B)' = A' \cdot B'$
- Minterm (mi) & Maxterm (Mi): Minterm ใช้ใน SOP (Sum of Products) เมื่อเอาต์พุตเป็น 1, Maxterm ใช้ใน POS (Product of Sums) เมื่อเอาต์พุตเป็น 0
- ตาราง K-Map 4x4: จัดลำดับตาม Gray Code ('00', '01', '11', '10') การรวมกลุ่ม 4 ช่อง (Quad) ตัดตัวแปรเปลี่ยนค่าได้ 2 ตัวแปร

1.2 วงจรบวก Full Adder & ริงล้น Overflow

- Sum: $Sum = A \oplus B \oplus Cin$
- Carry Out: $Cout = AB + Cin(A \oplus B)$
- Overflow Flag (V): $V = Cin(MSB) \oplus Cout(MSB)$ เกิดการล้นของเลขมีเครื่องหมาย 2's Complement (-128 ถึง +127)

1.3 ฟลิปฟลอป & วงจรนับ (Sequential Circuits)

- D Flip-Flop: $Q(t+1) = D$ (ถ้าค่า D เมื่อมีขอบ Clock)
- JK Flip-Flop: $Q(t+1) = J \cdot Q' + K' \cdot Q$ (J=1, K=1 เกิด Toggle สลับสถานะ)

- T Flip-Flop: $Q(t+1) = T \oplus Q$ (T=1 เกิด Toggle, T=0 Hold ค่าเดิม)
- Synchronous Counter: สัญญาณ Clock ต่อเข้าป้อนฟลิปฟล็อปทุกตัวพร้อมกัน ป้องกัน Propagation Delay สะสม

1.4 หน่วยควบคุม CPU (Control Unit: Hardwired FSM vs Microprogrammed ROM)

คุณสมบัติ	Hardwired FSM Control Unit	Microprogrammed ROM Control Unit
ฮาร์ดแวร์	เกตลอจิก, Decoders & Flip-Flops FSM	หน่วยความจำ ROM (Control Store) เก็บไมโครโค้ด
ความเร็ว	เร็วมาก (High Speed)	ช้ากว่าเล็กน้อย (ต้องอ่านแอดเดรส ROM)
ความยืดหยุ่น	ยากมาก (ต้องต่อสายไฟหรือแก้ Lay out ชิปใหม่)	ง่ายมาก (เพียงโปรแกรมข้อมูลไบนารีใน ROM ใหม่)
สถาปัตยกรรม	CPU แบบ RISC (เน้นความเร็ว)	CPU แบบ CISC (เน้นคำสั่งซับซ้อน)

1.5 สถาปัตยกรรมไปป์ไลน์ CPU (5-Stage Pipeline & Hazards)

- 5 สเตจไปป์ไลน์: IF (Instruction Fetch) → ID (Instruction Decode) → EX (Execute) → MEM (Memory Access) → WB (Writeback)
- Pipeline Hazards: Data Hazard (RAW) แก้ด้วย Data Forwarding/Bypassing, Control Hazard แก้ด้วย Branch Prediction / Flush



PART 2: สรุปเน้นสอบตอนบ่าย (เปิดตำรา - OPEN-BOOK EXAM)

2.1 สถาปัตยกรรมฮาร์ดแวร์ ATmega328P Microcontroller (Mazidi Ch 1–2)

- **Harvard Architecture:** แยกบัส Flash ROM 32KB และ Data SRAM 2KB ออกจากกัน ดึงคำสั่งและอ่านข้อมูลได้พร้อมกันใน 1 คาบ Clock
- **Register File:** 32 General Purpose Working Registers (R0 ถึง R31) โดยที่ R16-R31 สามารถใช้คำสั่ง **LDI** ได้โดยตรง
- **Status Register (SREG) 8 บิต:** [I | T | H | S | V | N | Z | C]
 - **Z (Zero):** เป็น 1 เมื่อผลลัพธ์เป็น 0 | **C (Carry):** เกิดตัวทศออกจาก MSB | **V (Overflow):** เลขมีเครื่องหมายล้น

2.2 พอร์ต I/O และคำสั่งจัดการบิต (Mazidi Ch 4)

รีจิสเตอร์	หน้าที่ (Function)	คำสั่งบิตสำคัญ	การทำงาน
DDRx	กำหนดทิศทาง (1=Output, 0=Input)	SBI PORTB, 5	เซตบิตที่ 5 ของ PORTB ให้เป็น 1 (ติดไฟ LED)
PORTx	ส่งเอาต์พุต / เปิด Pull-up	CBI PORTB, 5	เคลียร์บิตที่ 5 ของ PORTB ให้เป็น 0 (ดับไฟ LED)
PINx	อ่านค่าสัญญาณอินพุตจากพิน	SBIC PINC, 0	ข้าม 1 คำสั่งถ้าพิน PC0 เป็น 0 (สวิตช์กดลงดิน)

2.3 ชุดคำสั่งคำนวณและโปรแกรมภาษา AVR Assembly (Mazidi Ch 3, 5, 6)

หมวดคำสั่ง	คำสั่ง	รูปแบบ	การทำงาน
ย้ายข้อมูล	LDI / MOV / IN / OUT	LDI Rd, K / OUT A, Rr	โหลดค่าคงที่ K ลง Rd / ส่ง Rr ออก I/O Register A
คำนวณ ALU	ADD / SUB / SUBI / MUL	MUL Rd, Rr	คูณเลข 8 บิต ผลลัพธ์ 16 บิตเก็บบนคู่ R1 : R0
วนลูป & กระโดด	DEC / BRNE / BREQ / CALL	DEC Rd + BRNE label	ลดค่า Rd ที่ละ 1 และกระโดดกลับลูปจนกว่าจะเท่ากับ 0

2.4 ซอร์สโค้ดภาษา AVR Assembly (.asm) จากรายงานแล็บจริงสำหรับเปิดทำข้อสอบ



1. โค้ดไฟกะพริบพร้อมกันบน PORTB และ PORTD (AssemblyBlink1.asm)

```

;=====
; Title: AssemblyBlink1.asm
; Created: 11/22/2013
; Author: khatathapswatdipisal
; Description: Blink all LEDs on PORTB and PORTD simultaneously
;=====

.nolist
.include "m328pdef.inc"
.list

; Start at main after reset
rjmp main

main:
    ldi r16, 0xFF          ; Load 0xFF into R16 (all bits set to 1)
    out DDRB, r16          ; Configure Port B pins as Outputs
    out DDRD, r16          ; Configure Port D pins as Outputs
    ldi r16, 0x00          ; Clear R16 to 0x00

loop:
    com r16                ; Complement R16 (toggle bits between 0x00 and 0xFF)
    out PORTB, r16         ; Write R16 to Port B
    out PORTD, r16         ; Write R16 to Port D
    call MY_DELAY          ; Delay for simulation timing
    rjmp loop              ; Loop indefinitely

; Delay Subroutine
MY_DELAY:
    ldi r20, 8
L1:
    ldi r21, 200
L2:
    ldi r22, 250
L3:
    dec r22                ; Decrement R22
    brne L3               ; Loop if R22 != 0
    dec r21                ; Decrement R21
    brne L2               ; Loop if R21 != 0
    dec r20                ; Decrement R20
    brne L1               ; Loop if R20 != 0
    ret                   ; Return from subroutine

```

2. โค้ดไฟกะพริบสลับพอร์ต PORTB ติดเมื่อ PORTD ตับ (AssemblyBlink1_modified.asm)

```

;=====
; Title: AssemblyBlink1_modified.asm
; Created: 11/22/2013 (Modified: 2026-07-19)
; Author: khatathapswatdipisal
; Description: Alternate blinking: Port B ON when Port D is OFF
;=====

.nolist
.include "m328pdef.inc"
.list

; Start at main after reset
rjmp main

main:
    ldi r16, 0xFF      ; Load 0xFF into R16 (all bits set to 1)
    out DDRB, r16      ; Configure Port B pins as Outputs
    out DDRD, r16      ; Configure Port D pins as Outputs
    ldi r16, 0x00      ; Clear R16 to 0x00

loop:
    com r16             ; Complement R16 (toggle R16 between 0x00 and 0xFF)
    out PORTB, r16      ; Write R16 directly to Port B (PB0-PB7)

    mov r17, r16        ; Copy R16 to R17
    com r17             ; Complement R17 (so R17 is always the opposite of R16)
    out PORTD, r17      ; Write R17 to Port D (PD0-PD7)

    call MY_DELAY       ; Delay for simulation timing
    rjmp loop           ; Loop indefinitely

; Delay Subroutine
MY_DELAY:
    ldi r20, 8
L1:
    ldi r21, 200
L2:
    ldi r22, 250
L3:
    dec r22             ; Decrement R22
    brne L3            ; Loop if R22 != 0
    dec r21             ; Decrement R21
    brne L2            ; Loop if R21 != 0
    dec r20             ; Decrement R20
    brne L1            ; Loop if R20 != 0
    ret                ; Return from subroutine

```

3. โค้ดแปลง Hex 0-99 เป็น BCD 2 หลัก (Repeated Subtraction by 10)

```

.INCLUDE "M328PDEF.INC"
.ORG 0x0000

LDI R16, 0x45      ; ค่าอินพุต 0x45 = 69 ทศนิยม
CLR R17            ; R17 เก็บหลักสิบ (Tens)

BCD_LOOP:
    CPI R16, 10
    BRLO BCD_DONE
    SUBI R16, 10    ; ลบออกทีละ 10
    INC R17         ; เพิ่มหลักสิบ
    RJMP BCD_LOOP

BCD_DONE:
    SWAP R17
    OR R17, R16     ; รวมหลักสิบและหลักหน่วย
    OUT PORTB, R17  ; ส่งผลลัพธ์ BCD ออก PORTB

```

 4. โค้ดคำนวณผลรวมวนรูป $S = N + (N-1) + \dots + 1$

```

.INCLUDE "M328PDEF.INC"
.ORG 0x0000

LDI R16, 5         ; N = 5
CLR R17            ; ผลรวม S

SUM_LOOP:
    ADD R17, R16
    DEC R16
    BRNE SUM_LOOP

OUT PORTB, R17     ; ส่งผลลัพธ์ 15 (0x0F) ออก PORTB

```